

Congress Trading Signal

La couche données

Extraction, identification, enrichissement et validation honnête des déclarations
boursières du Congrès américain — Chambre et Sénat, 2020–2026

Alice Lemaire · 27 juin 2026

Résumé

Les membres du Congrès américain déclarent leurs transactions boursières sous le *STOCK Act* ; ces déclarations, exploitées par une stratégie de *copy-trading*, exigent au préalable une couche données propre, traçable et honnêtement validée. Ce rapport raconte la construction de cette couche, étape par étape — de l’acquisition des sources jusqu’à la table finale enrichie et validée. Le pipeline reconstruit **90 487 transactions** sur deux chambres et sept ans (2020–2026) : **81 646** pour la Chambre (32 676 électronique + 48 970 OCR) et **8 841** pour le Sénat (7 161 électronique + 1 680 papier OCR). Chaque transaction est rattachée à son auteur (**99,99 %** des lignes Chambre, **100 %** au Sénat), enrichie (ticker, secteur **GICS**→**ETF SPDR**, type d’actif, montant *midpoint*, ancienneté), aboutissant à la **table 12/12 champs**. La validation externe contre Quiver — **jamais réinjecté** — est conduite par **scope** (électronique / OCR / les deux) : au niveau transaction, on retrouve **98,2 %** des trades Quiver sur l’électronique de la Chambre, **68,3 %** sur l’OCR et **86,2 %** sur l’ensemble ; **99,4 %** au Sénat. Cette validation par scope corrige une idée reçue : **Quiver voit bien le papier de la Chambre** (les gros déposants-papier comme Khanna y figurent) — l’OCR Chambre est donc *validé en externe*, et la seule vraie limite est la **lecture OCR des dates manuscrites**. La correction est figée par un **golden octet-à-octet** (**125** fichiers Chambre, **76** Sénat) et des reproductions fonction-par-fonction. Chaque étape est exposée selon le même fil : le **problème** concret, la **façon dont il est résolu** (avec la fonction réelle), et le **résultat chiffré**.

1 Introduction & contexte

Depuis le **STOCK Act** (2012), les membres du Congrès américain doivent divulguer publiquement leurs transactions sur titres dans un *Periodic Transaction Report* (PTR), en principe sous **45 jours**. L’intuition de recherche — documentée académiquement (Ziobrowski *et al.* 2004/2011, Karadas 2019) — est que ces personnes, par leur appartenance à des commissions clés (Finance, *Armed Services*, Intelligence) ou leur exposition à l’information, produisent par leurs déclarations un **signal potentiellement exploitable**. Une stratégie de *copy-trading* consiste à répliquer ces transactions *après* leur publication (anti-*look-ahead* : on date chaque transaction par sa date de divulgation, jamais par une date postérieure connue).

Avant toute stratégie, il faut une **couche données** irréprochable : extraire fidèlement les déclarations des deux chambres, les rattacher à la bonne personne, les enrichir (ticker, secteur, montant) et les valider honnêtement. C’est l’objet de ce rapport. La stratégie et le backtest associés constituent un travail de recherche séparé, hors du périmètre traité ici.

2 Sources & acquisition

2.1 Les trois dates — et le principe anti-*look-ahead*

Une déclaration porte **deux dates** (parfois trois), qu’il ne faut pas confondre :

- `transaction_date` — **quand le trade a eu lieu**, lue *dans* le PTR (la ligne de transaction elle-même). C’est la date « économique ».
- `disclosure_date` — **quand la déclaration a été déposée**. Côté Chambre, c’est le `FilingDate`

de l'index XML (`house/digital.py`, l.262); côté Sénat, la dernière date de la liste eFD (`senate/digital.py`, l.100). C'est la date « publique » : avant elle, l'information n'existe pas pour un copieur.

- `notification_date` — **Chambre OCR seulement** : certains formulaires papier portent une date de notification distincte, conservée quand elle est lisible.

Pourquoi cette distinction est structurante. Une stratégie ne peut copier qu'*après* publication : on fenêtré et on date donc tout par la `disclosure_date`, jamais par la `transaction_date` (qui, elle, est antérieure et inconnue du marché au moment du dépôt). La clé naturelle d'une transaction (§4.8) **exclut volontairement** la `disclosure_date` : deux dépôts du même trade (un dépôt initial puis un amendement) doivent rester *une* transaction, pas deux. La divulgation sert d'axe temporel et d'*anti-look-ahead*, pas d'identité.

2.2 Où et comment on récupère les déclarations

Chambre — House Clerk. Source publique, sans barrière. Pour chaque année, le Clerk publie une archive ZIP contenant un **index XML** (`{an}FD.xml`) qui liste tous les dépôts : `DocID`, nom (`Last/First`), `StateDst`, `FilingType` et `FilingDate`. On lit cet index (`house/digital.py:load_ptr_index`), on filtre `FilingType=P` (les PTR), et on déduit l'URL du PDF de chaque dépôt (`resolve_pdf_path`). **C'est en ouvrant le PDF qu'on décide** s'il est lisible ou scanné (`build_manifest`, §4.2).

Sénat — eFD (*Electronic Financial Disclosure*). Plus fermé : un **garde-fou CSRF**. Il faut d'abord accepter un formulaire d'accord (`senate/digital.py:accept_agreement` : GET pour un `csrftoken`, puis POST `prohibition_agreement` avec `csrftoken` et l'en-tête `X-CSRFToken`). On interroge ensuite l'API de recherche (`fetch_ptr_list`, `report_types=[11]` = PTR); chaque dépôt y porte un `kind` — `ptr` (déclaration HTML électronique) ou `paper` (image .gif scannée). `fetch_report` récupère chaque déclaration. On reste « poli » (pause entre requêtes, aucun contournement) et on **cache tout sur disque** — un re-run ne re-télécharge rien.

2.3 D'où viennent les informations sur les élus (le référentiel)

Une déclaration ne donne qu'un *nom*. Pour savoir que « Buddy Carter » siège à la *Chambre*, qu'il est Républicain et dans telle commission, il faut un **référentiel des élus**. On utilise le jeu de données ouvert et maintenu *congress-legislators* (`unitedstates.github.io`), avec un **repli sur des YAML embarqués** dans le dépôt (`data/{chambre}/reference/legislators-*.yaml + committees-*.yaml`) — ainsi le pipeline reste reproductible hors-ligne. `common/reference.py:load_reference` en tire, pour chaque `bioguide_id` : la **chambre** (déduite du dernier mandat, `term.type`), le **parti**, et les **commissions** (`bio_to_committees`), avec un drapeau « commission clé » propre à chaque chambre. Ce référentiel est *partagé* par les deux chambres; seul le **matcher** `nom→bioguide` est spécifique (`house/identity.py`, `senate/identity.py`, §4.1).

2.4 Pourquoi l'API Claude Vision pour les scans

La moitié des déclarations de la Chambre, et tout le papier du Sénat, n'existent que comme **images** (PDF scannés, .gif) — souvent *couchées*, parfois *manuscrites*, avec des montants en **cases à cocher**. Un OCR classique (Tesseract) bute sur ces formulaires : il rend du texte brut sans structure, et lit très mal une page tournée ou une écriture à la main. On utilise donc l'**API Claude Vision** (modèle `claude-sonnet-4-6`), qui fait d'une pierre deux coups : elle *lit* l'image *et* la *structure* directement en transactions. Concrètement (`common/vision_ocr.py`, `house/ocr.py`, `senate/ocr_engine.py`) : l'appel force un `tool_use` sur l'outil `record_transactions` (le modèle *doit* répondre dans un schéma JSON strict — pas de texte libre à re-parser), avec **7 tentatives** et *backoff*, et un **cache versionné par prompt_sha** (un re-run ne re-paie que les lots en erreur : 0 appel si rien n'a changé). Le choix du modèle est assumé : `claude-sonnet-4-6` suffit ici (une sonde Opus n'a pas amélioré le point dur — les dates manuscrites — pour un coût $\approx \times 3$, §4.5).

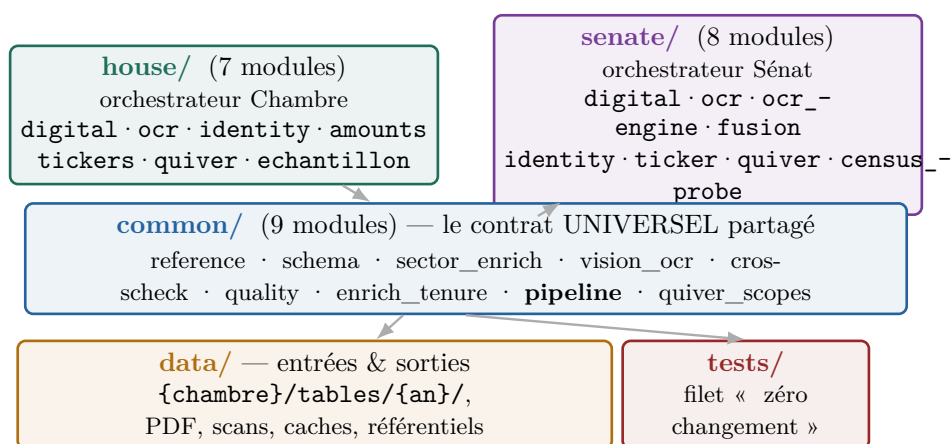
2.5 Quiver = vérification externe *uniquement*

Quiver Quantitative (agrégateur commercial) sert de **vérité-terrain indépendante**. Il n'est *jamais* réinjecté dans nos tables : on mesure si notre extraction concorde, on ne la complète pas. C'est une règle structurante du projet (validation détaillée en §6).

3 Architecture & méthodologie

3.1 Un contrat partagé + deux orchestrateurs jumeaux + données + tests

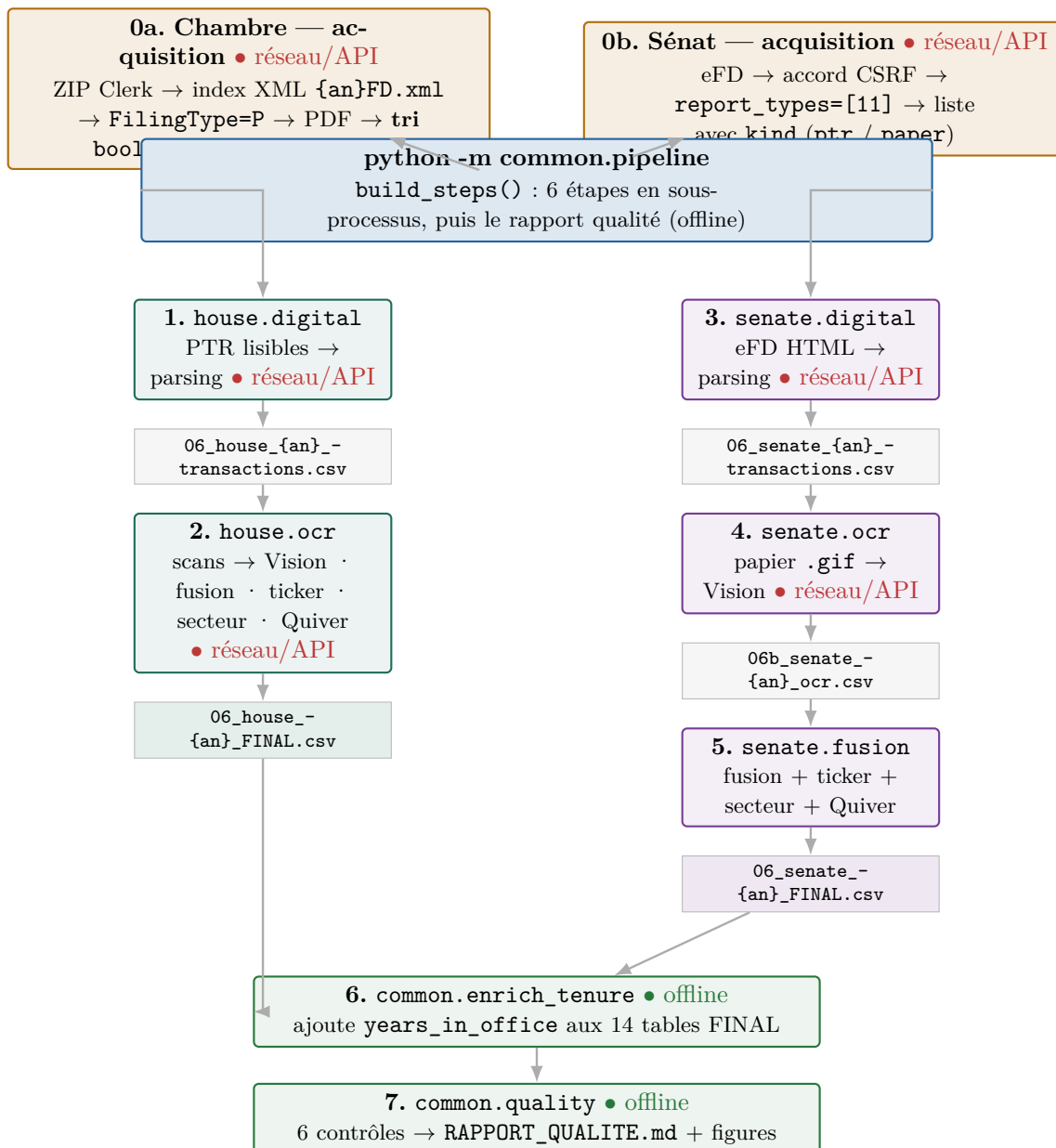
Le code est organisé en un **contrat universel partagé** (`common/`, 9 modules) que **deux orchestrateurs symétriques** (`house/` et `senate/`) utilisent. `common/` ne contient que ce qui est *vraiment* commun aux deux chambres (ou au pipeline, à la qualité, à l'ancienneté) ; tout ce qui *diffère* par chambre — le matcher d'identité, les montants, le ticker, l'OCR de production, la validation Quiver — vit dans `house/` et `senate/`, qui sont des **jumeaux**. Les données et les tests complètent l'ensemble.



La symétrie est volontaire : `house/` et `senate/` portent les *mêmes* rôles (`digital/ocr/identity/quiver...`), seules leurs implémentations diffèrent là où les sources diffèrent. La seule asymétrie de structure est `senate/fusion.py` (expliquée en §4.6) ; côté Chambre, la fusion vit dans `house/ocr.py`.

3.2 Le pipeline : le schéma à lire en premier

`common/pipeline.py` (fonction `build_steps`) enchaîne **6 étapes** en sous-processus ; un **7^e** traitement — le rapport qualité (`common/quality.py`) — se lance séparément, hors-ligne et en lecture seule. Le schéma ci-dessous montre, en plus des 6 étapes, l'**étape 0 d'acquisition & de tri** (§2) et la **fourche électronique/OCR** qui structure tout le reste.



3.3 La numérotation des fichiers : des étiquettes de rôle, pas l'ordre d'exécution

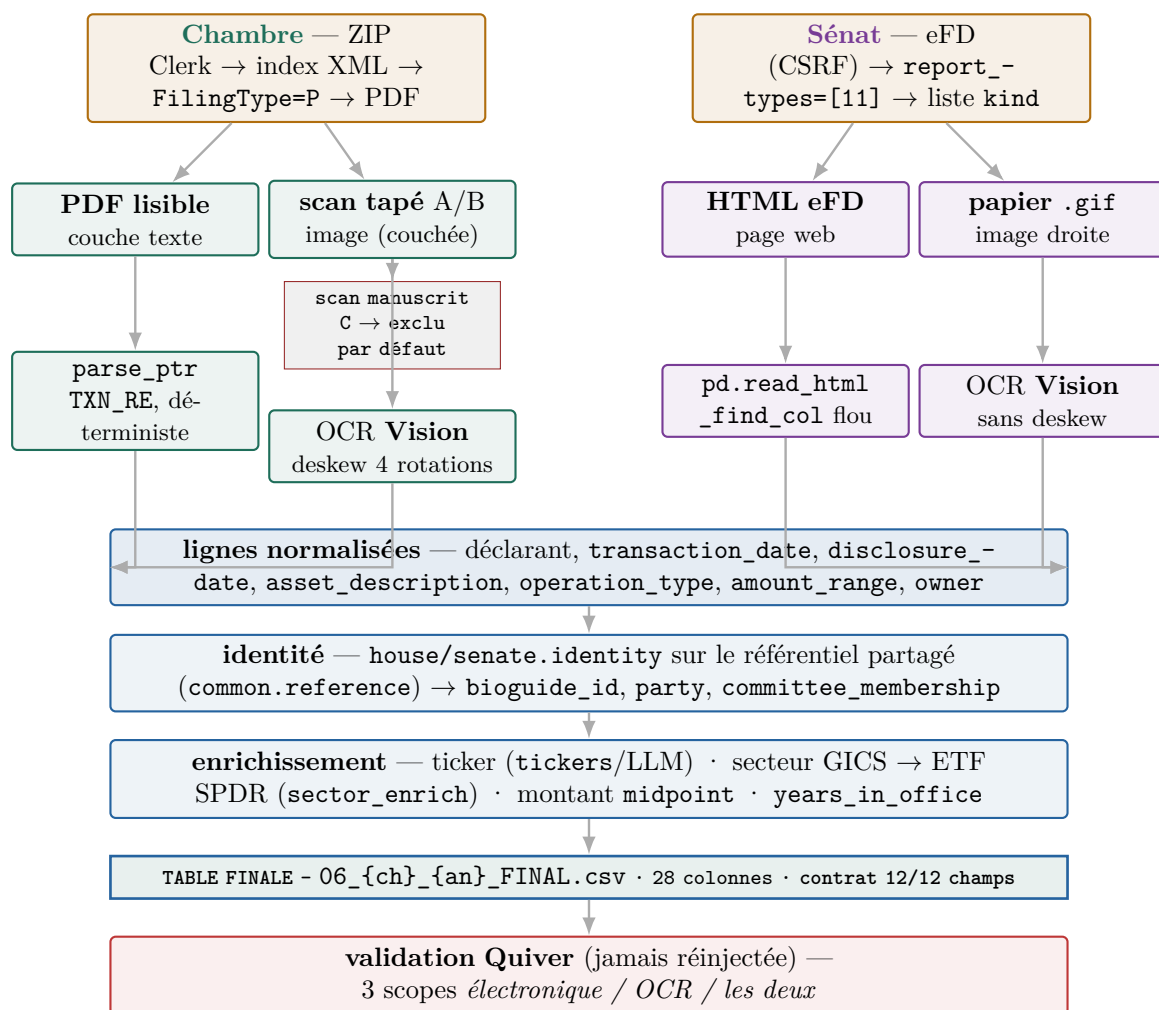
Sur le disque, le **préfixe** d'un fichier indique le **rôle** (la famille d'étape) de la table. C'est ce qui rend la couche données lisible sans documentation externe. **Attention** : le préfixe n'est *pas* strictement l'ordre d'exécution. Plusieurs tables d'un même numéro sont écrites par des modules différents, et certains numéros élevés sont produits *avant* des numéros plus petits — par exemple, au Sénat, 07_quiver_comparison (validation de la piste électronique) est écrit dès l'étape senate.digital, donc *avant* l'OCR 06b et la fusion 06..._FINAL. L'ordre réel est celui du pipeline (§3.2) ; les préfixes en sont une *approximation lisible*.

Préfixe	Rôle	Écrit par
03_	index des PTR (FilingType=P) dans la fenêtre (<i>Chambre</i>)	house.digital
04_	manifeste de tri : lisible / non_lisible / absent (<i>Chambre</i>)	house.digital
05_	échecs de parsing	*.digital
06_	table électronique (PTR lisibles / eFD HTML)	*.digital
06b_	table OCR (scans Chambre / papier Sénat)	house.ocr / senate.fusion
06c_	échecs OCR	house.ocr / senate.ocr
06d_	comparaison OCR↔Quiver (<i>Chambre</i>)	house.ocr
06_..._FINAL	électronique + OCR fusionnés , enrichis	house.ocr / senate.fusion
07_	comparaison Quiver de la piste électronique (par déposant)	*.digital / fusion
07b_	trades Quiver manquants, piste électronique (<i>Chambre</i>)	house.digital
07c-07f_	réconciliation Quiver 3 scopes (txn / champs / ticker / only-Quiver)	house.ocr / senate.fusion
07g_	décomposition Quiver par type d'actif	house.ocr / senate.fusion
07h_	décomposition Quiver par cluster de scan (<i>Chambre</i>)	house.ocr
08_	cross-check « semaine 1 » (<i>Chambre</i>)	house.digital
00_	tableaux de bord (statut par an / final) (<i>Sénat</i>)	senate.digital / fusion
_préfixe	caches & brouillons (_quiver*_cache, _scan_census_-547...)	divers

Pourquoi cette asymétrie ? La Chambre doit *calculer* elle-même le tri lisible/scanné (d'où 03/04 : ouvrir chaque PDF, tester sa couche texte), tandis que le Sénat reçoit le *kind déjà donné* par l'API eFD (pas de manifeste à produire). Les décompositions par cluster (07h) n'existent qu'à la Chambre, seule à disposer d'un census de scans (§4.5). Les réconciliations 07c-07g sont, elles, **symétriques** : les deux chambres les produisent.

4 Construction de la donnée, étape par étape

Cette section suit une transaction *dans l'ordre où le pipeline la construit* — de l'acquisition jusqu'à la ligne enrichie, validée, de la table finale. Le **schéma maître** ci-dessous en est le résumé visuel complet : il montre les **cinq formes** sous lesquelles une déclaration arrive (deux chambres, et selon le type de document), comment chacune est extraite, puis le tronc *commun* — identité, enrichissement, table finale, validation. Le récit qui suit déroule chaque étape : le **problème** concret, **comment on le résout** (avec la fonction réelle), le **résultat** chiffré. Les choix *révisés* en cours de route sont signalés par un encart ocre.



4.1 D’abord, savoir qui a déclaré : la résolution d’identité

Le problème. Une déclaration ne donne qu’un *nom libre* — « Hon. Earl L. Carter » ici, « Buddy Carter » là, « N. Pelosi » ailleurs. Le même élu apparaît sous plusieurs graphies au fil des années ; sans clé stable, on le compterait deux fois et l’on ne pourrait rattacher ni son parti ni ses commissions.

La clé stable : le `bioguide_id`. C’est l’**identifiant officiel unique** attribué à chaque membre du Congrès par le *Biographical Directory* (par ex. P000197 = Nancy Pelosi). Le résoudre une fois pour toutes rend chaque ligne rattachable, déduplicable et joignable au référentiel.

Comment on le gère. À partir du référentiel partagé (`common/reference.py`, §2.3), chaque chambre construit son **matcher** (`house/identity.py:make_matcher`, `senate/identity.py`), un index de noms volontairement *tolérant* : pour chaque élu, on indexe non seulement (nom, prénom) mais aussi le **surnom** (richard→rick, william→bill…), le second prénom, et le nom de famille. La cascade résout (nom, prénom) par priorité décroissante : (1) un **override manuel** pour les cas irréductibles (un seul est figé côté Chambre : « Nicholas V. Taylor » = Van Taylor, `_MANUAL_BIO`) ; (2) la **correspondance exacte** sur l’index ci-dessus ; (3) à défaut, le **repli par nom de famille unique**. `common/reference.py:enrich_identity` applique le matcher puis ajoute `party`, `committee_membership` et le drapeau de commission clé. Côté Sénat, le matcher **exclut les délégués** non-votants (sans quoi ils apparaîtraient « chez Quiver mais pas chez nous »).

Le résultat. **99,99 %** des lignes Chambre (81 642 / 81 646) et **100 %** du Sénat reçoivent un `bioguide_id`. Les 4 lignes sans identifiant sont *une* déclaration manuscrite de 2023 (« Ada Norah Henriquez »). On compte **256 bioguides pour 275 noms** (Chambre) : l’écart, ce sont précisément les variantes orthographiques d’un même élu ramenées au même identifiant — exactement ce que la résolution corrige.

4.2 Ensuite, trier les formats : électronique ou scanné ?

Le problème. Une déclaration arrive sous deux formes très différentes, qui n'appellent pas le même traitement : un **texte** exploitable directement (on peut le *parser*), ou une **image** qu'il faut « lire » par OCR. Se tromper de piste, c'est soit perdre des données, soit gaspiller des appels Vision sur un document déjà lisible.

Comment on le gère. Côté Chambre, `house/digital.py:build_manifest` ouvre chaque PDF avec `pdfplumber` et applique un test binaire *simple et déterministe* — `bool(text.strip())` : une couche de texte non vide $\Rightarrow$ **lisible** ; vide $\Rightarrow$ **non_lisible** (piste OCR) ; fichier absent $\Rightarrow$ **absent**. Le verdict est écrit dans `04_download_manifest.csv` et **pilote tout le reste** ; sa simplicité même fait sa fiabilité. Côté Sénat, le tri ne se calcule pas : il est *donné* par la liste eFD (`fetch_ptr_list`), où chaque dépôt porte déjà son *kind* — `ptr` (HTML) ou `paper` (`.gif`).

Le résultat. Ce tri d'apparence anodine est la **fourche** du pipeline : il explique la composition finale (Chambre 32 676 électronique + 48 970 OCR ; Sénat 7 161 électronique + 1 680 papier) et oriente chaque document vers l'une des sous-sections suivantes.

4.3 La piste électronique de la Chambre (PDF lisibles)

Le problème. Sur un PDF Clerk lisible, les transactions ne forment pas un tableau propre : une *même* transaction peut s'étaler sur plusieurs lignes, le montant déborder sur la ligne suivante, le ticker se cacher entre parenthèses et le libellé courir sur des lignes de continuation. Un simple « découpage par ligne » raterait la moitié des transactions.

Comment on le gère. `house/digital.py:run_year` orchestre la séquence : `load_ptr_index` lit l'index XML et produit `03_ptr_index.csv` ; `parse_ptr` fait le gros du travail — il **reconstitue** d'abord les lignes coupées (`_join_split_lines`, qui recolle un montant scindé en deux), détecte chaque transaction par l'expression régulière `TXN_RE` (un code P|S|E, deux dates, une fourchette \$), puis remonte de quelques lignes pour rattacher la description d'actif et le ticker. `join_identity` rattache le bioguide et les métadonnées, et `finalize` calcule la clé naturelle, l'`occurrence_index` et déduplique (§4.8). La sortie est `06_house_{an}_transactions.csv` ; les PDF lisibles ne donnant *aucune* ligne sont tracés dans `05_parse_failures.csv` — un échec est toujours visible, jamais silencieux. Le « taux de parsing » mesure justement la part des PDF lisibles qui ont rendu au moins une transaction.

Le résultat. **32 676** transactions électroniques, avec un taux de parsing très élevé (99–100 %).

Dans la source. Un PTR électronique de la Chambre se lit comme un tableau propre (cf. annexe 11.1) : « Amazon.com, Inc. — Common Stock (**AMZN**) [ST] · Purchase · 03/16/2018 · \$1 001–\$15 000 ». Le ticker est entre parenthèses, le type d'actif entre crochets — d'où le parsing déterministe.

4.4 La piste électronique du Sénat (HTML eFD)

Le problème. Le Sénat n'expose pas de PDF : une fois le **garde-fou CSRF** franchi (§2), il faut parser des **tableaux HTML** dont l'intitulé et l'ordre des colonnes *varient* d'une déclaration à l'autre.

Comment on le gère. `senate/digital.py:run_year` appelle `accept_agreement` (CSRF), puis `fetch_ptr_list` (`report_types=[11]`) et `fetch_report` (avec cache disque). `parse_electronic` lit alors les tableaux via `pd.read_html` — qui renvoie des *DataFrames* bruts — et c'est `_find_col` qui **retrouve les bonnes colonnes par appariement flou** (chercher une colonne dont le nom contient « ticker », ou « asset » et « name »), seule façon robuste face à des en-têtes irréguliers. L'identité, le ticker et la déduplication sont délégués à `senate/identity.py:enrich`. Sortie : `06_senate_{an}_transactions.csv`.

Le résultat. **7 161** transactions électroniques pour le Sénat.

Dans la source. Une déclaration eFD du Sénat est une page web (cf. annexe 11.1) : « **YUM** · Yum! Brands · Sale (Full) · \$1 001–\$15 000 · Joint », avec le ticker cliquable et la date de transaction — d'où la lecture par `pd.read_html`.

4.5 La piste scannée de la Chambre (OCR Claude Vision)

Le problème. Reste l'autre moitié : des **images**. Le census des 547 PDF scannés (`_scan_census_-547.csv`) en distingue trois familles : **A — tapé droit** (74 docs), **B — tapé mais l'image est couchée** (322 docs, le gros du volume), **C — manuscrit** (151 docs). Un modèle de vision lit mal une page tournée, et plus mal encore une date manuscrite.

Comment on le gère. Le moteur Vision (§2.4) redresse d'abord.

Hypothèse explorée puis révisée. Des skew. L'idée naïve — demander au modèle « de combien faut-il tourner ? » — échoue : il répond presque toujours « 90 ». À la place, on lui montre la page aux **4 orientations** (`ROT_CANDIDATES=[0, 90, 180, 270]`) et on lui fait reconnaître celle qui est droite — une tâche de reconnaissance, bien plus fiable qu'une estimation d'angle. La concordance OCR passe de **59,7 % à ≈65 %** ; la part résiduelle d'erreur tient à l'ambiguïté *intrinsèque* des dates manuscrites, pas à la rotation.

`house/ocr.py:run_ocr_year` extrait ensuite en JSON structuré (`tool_use` forcé `record_transactions`, `claude-sonnet-4-6`, 7 tentatives), avec le cache versionné par `prompt_sha`. Le ticker, absent des formulaires papier, est **ré-associé** en propageant les symboles vus en électronique, complété par une passe LLM. **Point d'architecture** : côté Chambre, la **fusion électronique+OCR**, le **ticker**, le **secteur** et la **validation Quiver** se font **ANNÉE PAR ANNÉE**, pendant le run (`run_ocr_year` écrit directement `06_house_{an}_FINAL.csv`), parce que le volume y est énorme.

Hypothèse explorée puis révisée. Échantillon d'abord. Avant le run complet des 547 PDF, `house/echantillon.py` a OCR-isé un **échantillon stratifié** des clusters A/B/C pour *mesurer* la qualité et durcir le prompt — plutôt que de découvrir les modes d'échec en production.

Hypothèse explorée puis révisée. Cluster C manuscrit exclu. Le cluster C est **exclu par défaut** (dates trop incertaines pour une stratégie datée), avec récupération ciblée des gros déposants. Une sonde **Opus vs Sonnet** sur C a confirmé qu'Opus **ne corrige pas le goulot** (les dates), pour un coût $\approx \times 3$ — on garde Sonnet, C reste exclu.

Le résultat. **48 970** lignes OCR. Le volume est très concentré : Rohit Khanna en représente **62,9 %** à lui seul, le top 3 (Khanna, McCaul, Harshbarger) **92,4 %** — de gros déposants qui déclarent exclusivement sur papier.

Dans la source. Un scan de la Chambre est un **formulaire à cases** (cf. annexe 11.1), souvent *couché* : une transaction = une ligne où l'on coche le type (Purchase/Sale), la date, et la **fourchette de montant** (une colonne à cocher) — d'où le besoin de skew puis de Vision.

4.6 La piste papier du Sénat — et pourquoi la fusion y est séparée

Le problème. Côté Sénat, le papier est marginal (5 sénateurs) mais bien réel : des images `.gif`, *droites* (pas de skew nécessaire). Surtout, contrairement à la Chambre, les données sont *peu nombreuses* — un dictionnaire de tickers construit année par année y serait trop pauvre.

Comment on le gère. `senate/ocr.py` (moteur figé `senate/ocr_engine.py`) lit les `.gif` par le *même* moteur Vision, et écrit `06b`. Puis — et c'est là l'**unique vraie asymétrie d'architecture** entre les deux chambres — `senate/fusion.py:main` procède en deux temps : **Étape 1**, fusion non destructrice *par année* (électronique + OCR, dédup sur (`natural_key_hash`, `occurrence_index`)) ; **Étape 2**, enrichissement sur le **corpus complet en une seule passe** — `senate/ticker.py:resolve_tickers` puis `common/sector_enrich.py:enrich_sectors` sur *toutes* les années concaténées.

Hypothèse explorée puis révisée. Per-année (Chambre) vs corpus (Sénat). La Chambre a assez de volume pour traiter chaque année isolément ; le Sénat, petit et chargé en papier, gagne à **mutualiser le dictionnaire de tickers sur tout le corpus** (un symbole vu une année sert aux autres). D'où un module `fusion.py` dédié, exécuté *après* l'assemblage de toutes les années — là où la Chambre fond et enrichit *pendant* chaque année.

Le résultat. **1 680** lignes papier, très concentrées sur Blumenthal (~73 %). C'est ce papier (obligations municipales, *munis*) qui explique la baisse de couverture ticker/secteur du Sénat en 2025–26 (§5).

Dans la source. Le papier du Sénat est un formulaire scanné (cf. annexe 11.1) : ex. « Microsoft, NASDAQ/OTC · 2/17/10 », fourchette de montant en colonnes cochées — comme la Chambre OCR, mais droit et sans symbole boursier imprimé (à ré-associer).

4.7 Donner un secteur à chaque ticker : GICS → ETF SPDR

Le problème. La stratégie visée ne copie pas action par action mais **secteur par secteur**, via des *ETF*. Un **ETF** (*Exchange-Traded Fund*) est un fonds coté en Bourse qui réplique tout un **panier** de titres — ici, *tout un secteur* d'un coup. Encore faut-il, pour chaque ticker déclaré, savoir **à quel secteur il appartient**. Le standard est la classification **GICS** (*Global Industry Classification Standard*), qui range chaque société cotée dans l'un de **11 secteurs** (Énergie, Matériaux, Industrie, Consommation discrétionnaire, Consommation de base, Santé, Finance, Technologie, Services de communication, Services aux collectivités, Immobilier). On mappe ensuite chaque secteur GICS **1 :1** vers son ETF de la famille *Select Sector SPDR* (un ETF liquide par secteur) :

Energy→XLE · Materials→XLB · Industrials→XLI · Consumer Discretionary→XLY · Consumer Staples→XLP · Health Care→XLV · Financials→XLF · Information Technology→XLK · Communication Services→XLC · Utilities→XLU · Real Estate→XLRE

Comment on le gère. `common/sector_enrich.py:enrich_sectors` résout en cascade hybride, en **tranchant qui a tranché** (`sector_source`) : (1) `resolve_yfinance` — secteur *factuel* via `yfinance` ; (2) `repli_resolve_llm` — pour les tickers que `yfinance` ignore, un LLM (`claude-sonnet-4-6`) qui ne répond que dans l'énumération GICS stricte *et* pose un drapeau `is_listed` (pour ne *jamais* attribuer un secteur à un actif non coté) ; (3) overrides manuels (`audit adversarial`). La table `GICS_TO ETF` fait la dernière marche.

Hypothèse explorée puis révisée. *Ticker en trois voies.* Le ticker lui-même est résolu en trois voies ajoutées par échecs successifs : symbole **explicite** (déjà imprimé), **dictionnaire électronique** (nom → symbole vu ailleurs), puis **LLM** avec filtre actions — pour ne jamais tickeriser une obligation ou une muni.

« **Couverture** » — **ce que le mot veut dire**. La couverture ticker/secteur est un **taux de remplissage**, pas un taux d'exactitude : les actifs **non cotés** (obligations, munis, fonds privés) n'ont *légitimement* ni ticker ni secteur — il n'existe pas de symbole à leur attribuer. Une couverture < 100 % reflète donc surtout la *part d'actifs non cotés*, pas un défaut d'extraction.

Le résultat. Secteur renseigné à **83,2 %** (Chambre) et **62,1 %** (Sénat) ; ticker à 85,3 % / 71,4 % ; montants ramenés à leur milieu (`house/amounts.py:amount_midpoint`) ; ancienneté `years_in_office` à **100 %** sur les deux chambres (calculée hors-ligne et *appendue* en dernière étape, `common/enrich_tenure.py`, octet-préservant).

4.8 La table finale : le contrat « 12/12 »

Le liant — le cœur partagé. Ce qui assemble tout sans rien perdre, c'est `common/schema.py` : une **clé naturelle** de **7 champs** (`chamber`, `declarant_name`, `transaction_date`, `asset_description`, `operation_type`, `amount_range`, `owner`). Elle exclut *volontairement* le ticker — ajouté *après*, à l'enrichissement — et la `disclosure_date` (§2) : les inclure rendrait la clé instable. Cette clé est numérotée par `occurrence_index` (le rang d'un lot répété *dans un même dépôt*).

Hypothèse explorée puis révisée. *Déduplication.* Dédupliquer sur la seule clé naturelle écraserait des **lots multi-comptes réels** (Khanna déclarant plusieurs fois le même titre via plusieurs comptes). `add_occurrence_index + dedup_canonical` retirent les vrais doublons cross-dépôt mais **préservent** ces lots — *dédup non destructrice* sur le couple (`natural_key_hash`, `occurrence_index`).

Le contrat. La table FINAL compte **28 colonnes** (27 d'assemblage + `years_in_office`). Les **12 champs « métier »** exigés sont *exactement* la constante `schema.CORE_FIELDS` :

`declarant_name · chamber · party · committee_membership · transaction_date · disclosure_date · ticker · sector_gics · etf_proxy · operation_type · amount_midpoint · asset_type`

Nuance d'honnêteté — « présents » n'est pas « sans valeurs manquantes ». Les 12 champs sont **présents** dans le schéma des deux chambres. Mais trois ont des **trous légitimes**, par nature et non par défaut d'extraction : `ticker/sector_gics/etf_proxy` sont vides pour les actifs non cotés ; et `committee_membership` est un **snapshot courant**, pas l'historique daté. Les champs structurellement toujours renseignables (nom, chambre, parti, dates, opération, montant) sont, eux, quasi-complets. Les taux de remplissage exacts (par champ et par an) sont en annexe 11.4.

5 Le rapport de qualité

Le problème. Une table « pleine » n’est pas une table *saine* : les dates peuvent être incohérentes, les délais légaux non respectés, la couverture concentrée sur quelques personnes.

Comment on l’obtient. `common/quality.py` (`build_report`) calcule **six contrôles** et génère 4 figures — **en lecture seule des FINAL, sans aucun appel API** (matplotlib Agg). (a) **Cohérence des dates** (`disclosure ≥ transaction`) : 99,8 % Chambre, 100,0 % Sénat. (b) **Délai légal** : 86,9 % (Chambre) et 84,9 % (Sénat) des dépôts dans les 45 jours (médiane **28 j**). (c) **Montants** : médiane **8 000 \$**, quartile haut **32 500 \$**. (d) **Couverture par membre** : 320 déposants, dont **206** avec ≥ 10 transactions (éligibles) et **150** actifs sur ≥ 3 années. (e) **Achats sans sortie déclarée** : 22,5 % (Chambre) / 39,6 % (Sénat). (f) **Validation externe Quiver** par scope — détaillée en §6, agrégée ici depuis les tables 07c/07g/07h figées (`quality.quiver_validation`). Le détail par an est en annexe 11.4.

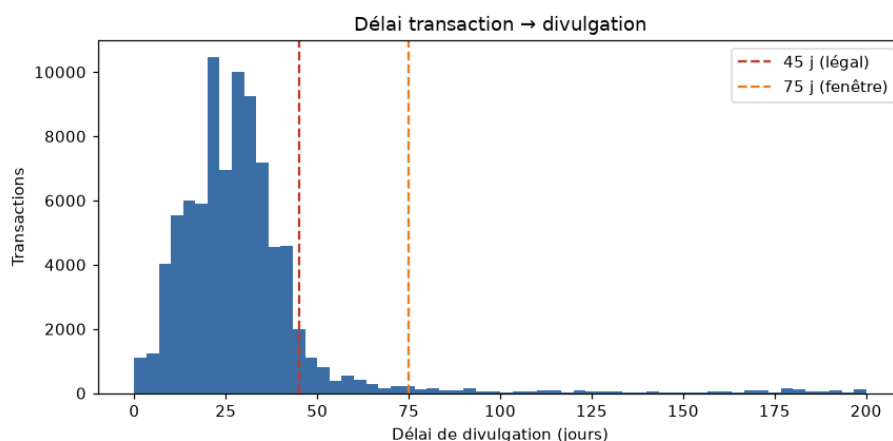


FIGURE 1 – Délai disclosure — transaction (0–200 j), lignes 45 j (légal) / 75 j (fenêtre). Générée par `common/quality.py` — lecture seule, aucun appel API.

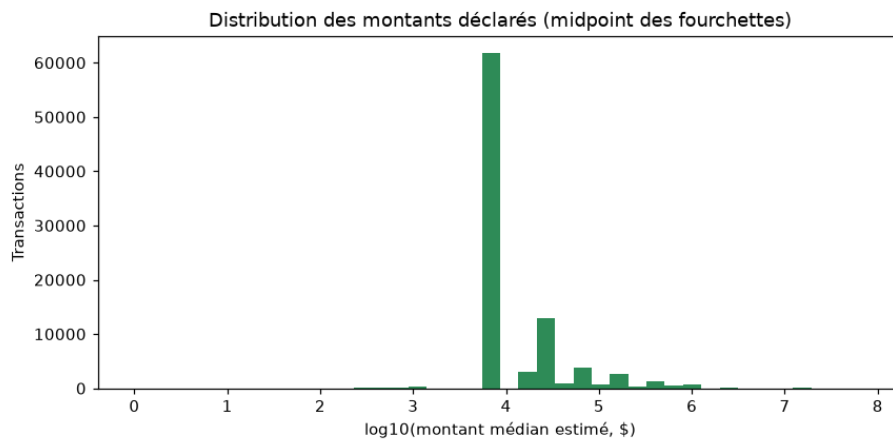


FIGURE 2 – Distribution de `amount_midpoint` en échelle $\log_{10}$. Générée par `common/quality.py`.

La couverture par an — et la baisse Sénat 2025–26. Le global masque une dynamique : au Sénat, la couverture ticker chute à **45,4 %** (2025) et **43,7 %** (2026), contre $\sim 80\%$ les années antérieures ; le secteur suit. **Pourquoi ?** La part d’obligations municipales (non cotées, donc `ticker=None` légitimement, ex. Blumenthal) y augmente fortement. Ce n’est pas une dégradation d’extraction mais un changement de composition des actifs. La fiabilité des dates OCR est suivie par `date_confidence` (fenêtre 75 j, colonne *OCR seulement*) : 95,3 % plausibles côté Chambre — le creux 2021 (79,6 %) est entièrement Diana Harshbarger (manuscrit dense), anomalie localisée, non systémique.

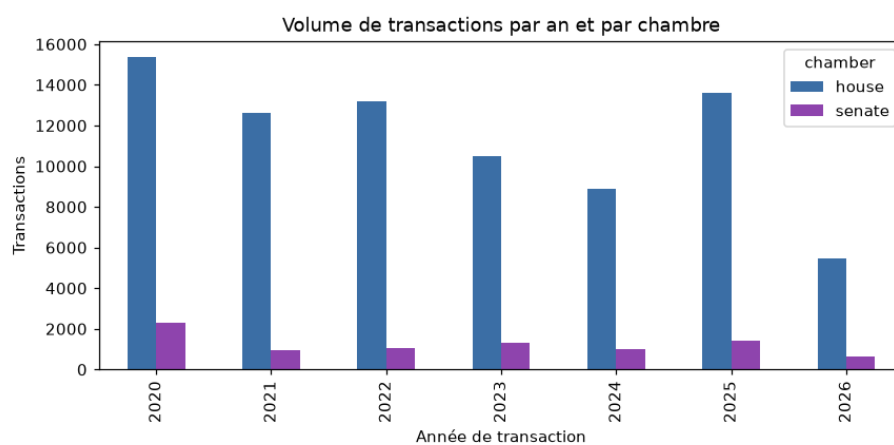


FIGURE 3 – Transactions par **année de transaction** et par chambre. Générée par `common/quality.py`.

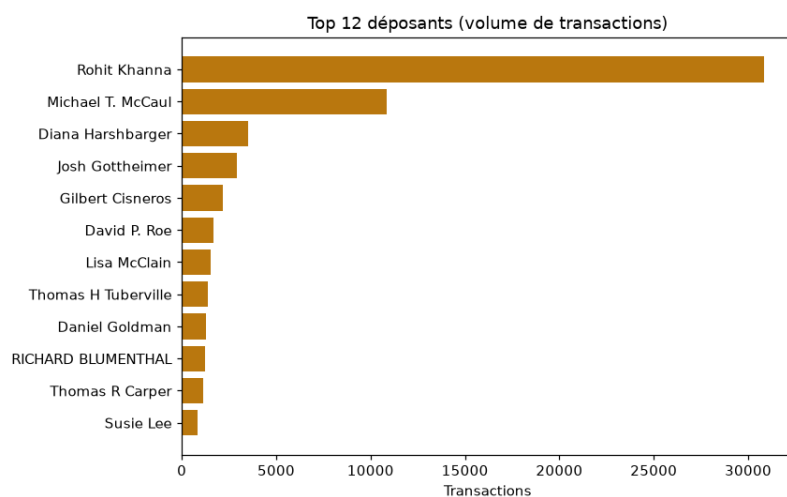


FIGURE 4 – Top 12 déposants par **nombre de transactions**. Générée par `common/quality.py` — la domination des déposants-papier (Khanna, McCaul) y est visible.

Hypothèse explorée puis révisée. Fenêtre de plausibilité. Le STOCK Act vise ≈ 45 j ; on tolère **75 j** (*date_confidence*) pour absorber le bruit OCR. C'est une heuristique de plausibilité *sur l'OCR seulement*, pas une vérité-terrain — la vérité-terrain, c'est Quiver (§6).

6 La validation externe (Quiver)

Le problème. Comment savoir si l'extraction est juste, *sans* se contenter de se croire ?

Comment on le gère — la validation par scope. On confronte le FINAL à Quiver au niveau transaction (clé *bioguide* | *ticker* | *date* | *sens*, la date appariée étant le *Traded* de Quiver), **sans jamais le réinjecter**, et — c'est l'apport central — **par scope** : *électronique seul*, *OCR seul*, et *les deux ensemble* (`common/quiver_scopes.py:reconcile_scopes`). La **couverture** = part des trades Quiver qu'on retrouve ; *only_ours* = nos trades absents de Quiver ; *only_quiver* = trades Quiver qu'on n'a pas. Une seconde fonction (`match_breakdown`) décompose chaque appariement en *exact_match* (même trade, même date), *date_mismatch* (bon trade, notre date diffère), *no_match* (absent de Quiver) et *non_equity* (muni/obligation, **hors périmètre Quiver**), par **type d'actif** et par **cluster de scan**.

Couverture par scope (transaction-niveau, toutes années) :

Scope	Chambre	Sénat	lecture
électronique (<i>digital</i>)	98,2 %	99,4 %	le parsing déterministe est quasi exact
OCR seul (<i>ocr</i>)	68,3 %	<i>vide</i>	cf. les deux constats ci-dessous
les deux (<i>both</i>)	86,2 %	99,4 %	union dédoublée des trades

Constat 1 — Quiver ne couvre que les ACTIONS. La décomposition par type d'actif le montre : sur les Stock, Quiver *possède* le trade dans **87,5 %** des cas (Chambre) et **84,2 %** (Sénat) ; mais les *non_equity* (obligations d'État, *mutual funds*, et surtout **obligations municipales**) sont **hors de son périmètre**. Un *non_equity* non apparié n'est ni une erreur de notre côté, ni un défaut : c'est un actif que Quiver *ne suit pas*. C'est toute l'explication du Sénat : son OCR papier est **surtout du non-coté** (munis de Blumenthal), d'où un scope OCR « vide » côté Quiver — *pas* parce que Quiver ignorerait le papier, mais parce qu'il ignore les *munis*. Les actions du Sénat, elles, sont très bien couvertes.

Constat 2 — la découverte : Quiver voit le papier de la Chambre. Contrairement à ce qu'on croyait, Quiver **possède bien** les gros déposants-papier de la Chambre (Khanna, McCaul. . .). L'OCR de la Chambre est donc *validé en externe*, et la décomposition **par cluster de scan** situe précisément notre limite :

Cluster (Chambre)	Quiver a le trade	date juste	lecture
A — tapé droit	88,0 %	85,7 %	lecture fiable
B — tapé couché	77,9 %	68,2 %	le skew suffit, dates correctes
C — manuscrit	35,3 %	37,2 %	<i>point dur</i> → cluster exclu

« Date juste » = *exact_match* parmi les trades que Quiver possède. La leçon : même quand Quiver a le trade, c'est notre **lecture OCR de la date** qui décroche sur le manuscrit (37 %) — d'où l'exclusion du cluster C (§4.5), *pas* une cécité de Quiver au papier.

Le résultat. Aucune erreur d'extraction systématique : sur l'électronique, la concordance est quasi parfaite (98–99 %) ; sur l'OCR, les écarts s'expliquent intégralement par (i) le non-coté hors périmètre Quiver et (ii) la date des scans manuscrits. *only_ours* (nos trades absents de Quiver) est donc surtout du **non-coté** et du **manuscrit**, pas du bruit. `common/crosscheck.py` matérialise ce statut par déposant (`quiver_validable` / `ocr_unique` / `digital` ; détail en annexe 11.4).

7 Assurance qualité & reproductibilité

Le problème. Le pipeline n'est **pas rejouable hors-ligne** (le scraping exige le réseau, l'OCR l'API) ; rejouer une année électronique produirait une table *vide* (les PDF lisibles ont été parsés puis écartés).

Comment on le gère. On prouve la correction par **reproduction depuis les colonnes figées**, preuve plus forte car déterministe : (i) un **golden octet-à-octet** — empreintes SHA-256 de toutes les sorties, **125** fichiers Chambre et **76** Sénat (`check_golden.py` / `senate_check_golden.py` doivent afficher « ZÉRO ÉCART ») ; (ii) des **reproductions fonction-par-fonction** (`test_senate_repro` reconstruit `natural_key_hash` 8 841/8 841, l'identité 67/67, le ticker ; côté Chambre `test_schema/test_amounts_tickers/` `test_identity/test_tenure`) ; (iii) `test_vision_sha/test_incremental` (le `prompt_sha` est stable, un 2^e run OCR fait 0 appel) ; (iv) `audit_metrics.py`, qui recompute toutes les métriques et *assure* les invariants — la source de vérité chiffrée de ce rapport. Les réconciliations Quiver 07c-07h sont déterministes (tri à départage stable), donc figées dans le golden au même titre que les tables.

8 Limites (honnêteté)

- **Cluster C manuscrit exclu par défaut** : la limite n'est pas la couverture Quiver (qui a le trade) mais notre **lecture OCR des dates manuscrites** (date juste 37 %) — choix assumé.
- **ticker/sector_gics/etf_proxy structurellement absents** pour munis et obligations (actifs non cotés) — par nature, pas par défaut. C'est aussi ce qui rend une partie de l'OCR **hors périmètre Quiver** (qui ne suit que les actions).
- **Commissions non « point-in-time »** : `committee_membership` est un *snapshot* courant, pas l'historique daté.
- **date_confidence = heuristique binaire** (fenêtre 75 j), **colonne OCR seulement** ; ce n'est pas une vérité-terrain (Quiver l'est).
- **Baisse de couverture Sénat 2025–26** : réelle mais *de composition* (montée des munis), pas d'extraction.

9 Périmètre & suite

La **couche données** décrite ici est livrée : 90 487 transactions extraites, identifiées, enrichies (table 12/12), validées et reproductibles. La **stratégie et le backtest** associés (copy-trading actions puis ETF sectoriels) constituent un travail de recherche séparé, hors-périmètre de ce rapport.

10 Conclusion

Sur deux chambres et sept ans, le pipeline reconstruit **90 487** transactions à partir des sources officielles, en traitant honnêtement les deux pistes (électronique déterministe *et* scannée par OCR Vision). Chaque ligne est identifiée (99,99 / 100 %), enrichie (table 12/12 sur les deux chambres) et validée contre Quiver sans jamais le réinjecter, **par scope** : 98,2 % sur l'électronique Chambre, 68,3 % sur l'OCR, 86,2 % sur l'ensemble ; 99,4 % au Sénat. Cette validation par scope a corrigé une idée reçue — Quiver *voit* le papier de la Chambre — et a localisé notre seule vraie limite, la **lecture des dates manuscrites**, tout en montrant que les écarts du Sénat viennent du **non-coté hors périmètre Quiver**, non d'erreurs. La correction est figée (golden octet-à-octet, 125/76) et reproductible. Surtout, le rapport assume ses nuances — « présents » n'est pas « sans manquants », la couverture Sénat 2025–26 baisse pour des raisons de composition d'actifs. C'est une couche données sur laquelle une stratégie peut être bâtie *en connaissant ses limites*.

11 Annexes

11.1 Annexe A — aperçu des sources

À quoi ressemble la matière première des **quatre pistes** (deux chambres × électronique/OCR). Captures des plateformes publiques (*House Clerk*, Sénat *eFD*) et rendu d'un scan du dépôt.

Owner Asset	Transaction Date	Notification Amount	Cap. Gains > \$200?
Amazon.com, Inc. - Common Stock (AMZN) [ST]	03/16/2025	\$1,001 - \$15,000	☐
Apple Inc. - Common Stock (AAPL) [ST]	03/16/2025	\$1,001 - \$15,000	☐
AT&T Intellectual Property [ST]	03/16/2025	\$1,001 - \$15,000	☐

(a) **Chambre — électronique.** PTR (*House Clerk*) : tableau texte, ticker entre parenthèses (AMZN), type d'actif entre crochets.

Transaction Date	Transaction Type	Transaction Amount
03/16/2025	BUY	\$1,001 - \$15,000
03/16/2025	BUY	\$1,001 - \$15,000
03/16/2025	BUY	\$1,001 - \$15,000

(b) **Chambre — OCR.** PDF scanné, ici *couché* : formulaire à cases, redressé par deskew avant lecture Vision.

Transaction Date	Owner	Ticker	Asset Name	Asset Type	Type	Amount	Comment
02/18/2025	Joint	YUMC	Yum! Brands Company: Yum (Louisville, KY) Description: Food	Other	Sale (Full)	\$1,001 - \$15,000	--

(c) **Sénat — électronique.** Page web eFD : ticker cliquable (YUM), type d'opération et montant — lue par `pd.read_html`.

Transaction Date	Transaction Type	Transaction Amount
03/16/2025	BUY	\$1,001 - \$15,000
03/16/2025	BUY	\$1,001 - \$15,000
03/16/2025	BUY	\$1,001 - \$15,000

(d) **Sénat — OCR.** Formulaire papier scanné (eFD *media*) : lignes à cocher, fourchettes de montant en colonnes.

11.2 Annexe B — modules & fonctions clés

Pour chaque module : son rôle, et ses **fonctions centrales** (les helpers privés mineurs sont omis). Le code vit dans **24 modules** (common/ 9 + house/ 7 + senate/ 8); l'orchestration se lit dans `common/pipeline.py:build_steps`, puis `house/digital.py:run_year`, `house/ocr.py:run_ocr_year`, `senate/digital.py:run_year` et `senate/fusion.py:main`.

Module	Rôle	Fonctions clés
common/ — le contrat universel partagé (9)		
reference.py	référentiel des élus	load_reference · Reference · enrich_identity · add_years_in_office · bio_to_committees

Module	Rôle	Fonctions clés
<code>schema.py</code>	clé naturelle + dédup	<code>natural_key_hash</code> (SHA-256 de 7 champs) · <code>add_occurrence_index</code> · <code>dedup_canonical</code> (non destructrice) · <code>CORE_FIELDS</code>
<code>sector_enrich.py</code>	secteur GICS → ETF	<code>resolve_yfinance</code> · <code>resolve_llm</code> (<code>is_listed</code>) · <code>enrich_sectors</code> (+ <code>sector_source</code>) · <code>GICS_TO ETF</code>
<code>vision_ocr.py</code>	OCR Vision de référence	<code>detect_rotation</code> (deskew 4 rotations) · <code>pdf_to_b64_images</code> · <code>VisionExtractor.prompt_sha</code> · <code>.extract_cached</code>
<code>crosscheck.py</code>	statut par déposant	<code>per_filer_status</code> · <code>add_external_counts</code> (Kadoda/HSW) · <code>summary</code>
<code>quality.py</code>	rapport qualité (6 contrôles)	<code>load_final</code> · <code>date_coherence</code> · <code>delay_buckets</code> · <code>amount_distribution</code> · <code>coverage_per_member</code> · <code>unmatched_purchase_rate</code> · <code>quiver_validation</code> · <code>build_report</code>
<code>enrich_tenure.py</code>	ancienneté	<code>add_years_in_office</code> (append <code>years_in_office</code> octet-préservant) · <code>main</code>
<code>pipeline.py</code>	orchestrateur	<code>build_steps</code> (séquence 6 étapes) · <code>main</code> (sous-processus, -dry-run)
<code>quiver_scopes.py</code>	validation Quiver multi-scopes	<code>reconcile_scopes</code> (digital/ocr/both → 07c-07f) · <code>match_breakdown</code> (par actif & cluster → 07g/07h)
house/ — orchestrateur Chambre (7)		
<code>digital.py</code>	PTR électroniques	<code>load_ptr_index</code> (XML, <code>FilingType=P</code>) · <code>resolve_pdf_path</code> · <code>build_manifest</code> (tri lisible/non) · <code>parse_ptr</code> (<code>TXN_RE</code> , <code>_join_split_lines</code>) · <code>finalize</code> · <code>validate_quiver</code> · <code>run_year</code>
<code>ocr.py</code>	scans → FINAL (par an)	<code>run_ocr_year</code> (Vision + fusion + ticker + secteur + Quiver 07c-07h) · <code>_infer_asset_type</code>
<code>identity.py</code>	matcher Chambre	<code>make_matcher</code> (override→exact+surnoms→nom unique) · <code>_MANUAL_BIO</code>
<code>amounts.py</code>	montant & opération	<code>amount_midpoint</code> (milieu de fourchette) · <code>operation_type_from_code</code> (P/S/E) · <code>HOUSE_OCR_AMOUNT_MAP</code>
<code>tickers.py</code>	ticker + type d'actif	<code>explicit_ticker</code> · <code>recover_ticker</code> · <code>infer_asset_type</code> (par chambre)
<code>quiver.py</code>	réconciliation Quiver	<code>reconcile</code> · <code>norm_ticker</code> (robuste NaN/vide)
<code>echantillon.py</code>	pilote OCR stratifié	<code>run_echantillon</code> · <code>compare_quiver</code>
senate/ — orchestrateur Sénat (8)		
<code>digital.py</code>	eFD électronique	<code>accept_agreement</code> (CSRF) · <code>fetch_ptr_list</code> (<code>report_types=[11]</code>) · <code>fetch_report</code> · <code>parse_electronic</code> (<code>pd.read_html</code> , <code>_find_col</code>) · <code>run_year</code>
<code>ocr.py</code>	papier .gif → 06b	<code>discover_index</code> · <code>ocr_one</code> (un .gif → Vision) · <code>build_table</code>
<code>ocr_engine.py</code>	moteur OCR figé	<code>extract_report</code> · <code>normalize</code> · <code>_infer_asset_type</code>
<code>fusion.py</code>	fusion + enrich corpus	<code>main</code> (Étape 1 fusion/an · Étape 2 enrichissement corpus + Quiver 07c-07g) · <code>date_confidence</code>
<code>identity.py</code>	matcher Sénat	<code>load_reference</code> · <code>make_matcher</code> (désambiguïsation chambre, exclut délégués) · <code>enrich</code>
<code>ticker.py</code>	ticker 3 voies	<code>resolve_tickers</code> (explicit→dict→LLM) · <code>llm_resolve_tickers</code>
<code>quiver.py</code>	réconciliation Quiver	<code>reconcile</code> · <code>norm_ticker</code> · <code>norm_sense</code>
<code>census_probe.py</code>	Phase 0 (census)	<code>parse_electronic</code> · <code>main</code> (census + sondage parser)

Les `__init__.py` (exports de paquet) et `tests/regression/` (golden + reproductions) complètent le dépôt. Le secteur (`common/sector_enrich`) et la clé (`common/schema`) sont importés en direct par les deux orchestrateurs — d'où leur place dans `common/`.

11.3 Annexe C — le schéma FINAL (28 colonnes)

#	Colonne	Sens	Remplie par
1	bioguide_id	identifiant officiel du membre	identity
2	declarant_name	nom tel que déposé	parse / OCR
3	chamber	house / senate	enrich_identity
4	party	parti	Reference.party
5	state_district	état + district	03_ptr_index
6	committee_membership	commissions (liste)	Reference.committees
7	committees_key_flag	commission « clé » ?	reference
8	transaction_date	date de la transaction	parse / OCR
9	disclosure_date	date de divulgation (dépôt)	03_ptr_index / eFD
10	ticker	symbole boursier	tickers / ticker
11	asset_description	libellé de l'actif	parse / OCR
12	asset_type	type d'actif	infer_asset_type
13	sector_gics	secteur GICS	sector_enrich
14	etf_proxy	ETF SPDR du secteur	GICS_TO_ETF
15	operation_type	Purchase / Sale / ...	operation_type_from_code
16	amount_range	fourchette déclarée	parse / OCR
17	amount_midpoint	milieu de la fourchette	amount_midpoint
18	amount_split_flag	lot multi-actifs ?	finalize
19	owner	propriétaire (self/spouse...)	normalize
20	doc_id	identifiant du PTR	03_ptr_index
21	source_url	URL de la source	03_ptr_index
22	natural_key_hash	clé de dédup (7 champs)	schema
23	occurrence_index	rang du lot intra-dépôt	add_occurrence_index
24	provenance	*-electronic / *-ocr	pipeline
25	date_confidence	plausible / implausible (75 j)	date_confidence
26	ticker_source	explicit / elec_dict / llm / none	tickers / ticker
27	sector_source	yfinance / llm / manual / none	sector_enrich
28	years_in_office	ancienneté à la date du trade	enrich_tenure

L'ordre exact diffère légèrement entre Chambre et Sénat (date_confidence plus tôt au Sénat); le contenu garanti — les 12 champs « métier » — est identique. Les 27 colonnes d'assemblage (HOUSE_FINAL_SCHEMA / SENATE_FINAL_SCHEMA) + years_in_office = 28.

11.4 Annexe D — métriques vérifiées (recalculées depuis les FINAL)

Lue après le rapport, cette annexe dit si l'extraction a bien marché. Tous les chiffres sont recomputés depuis les tables FINAL par tests/regression/audit_metrics.py, common/quality.py et les réconciliations Quiver figées (07c-07h).

D.1 — Volumes par an.

An	Chambre				Sénat			
	élec.	OCR	FINAL	dépos.	élec.	OCR	FINAL	sén.
2020	6 886	8 791	15 677	109	1 706	255	1 961	33
2021	5 457	6 816	12 273	120	1 098	281	1 379	35
2022	3 601	10 900	14 501	110	919	182	1 101	27
2023	4 161	6 329	10 490	100	1 062	97	1 159	30
2024	2 694	6 277	8 971	97	946	84	1 030	24
2025	7 577	6 067	13 644	107	943	478	1 421	31
2026	2 300	3 790	6 090	84	487	303	790	22
Tot.	32 676	48 970	81 646	256*	7 161	1 680	8 841	64*

* bioguides *uniques* sur 7 ans (la somme annuelle compte un élu chaque année).

D.2 — Couverture des champs enrichis, par an (% renseigné).

An	Chambre				Sénat			
	ticker	sect.	a.type	comm.	ticker	sect.	a.type	comm.
2020	86,5	83,4	91,0	64,9	88,1	80,5	96,3	34,5
2021	87,3	85,8	90,1	78,7	80,9	70,1	94,2	54,3
2022	87,1	85,2	91,0	89,2	79,7	65,3	96,1	78,3
2023	78,9	76,6	88,6	96,8	77,7	69,5	96,3	71,4
2024	81,3	78,3	87,4	93,6	68,0	59,2	100	77,8
2025	87,8	86,3	94,8	97,6	45,4	37,2	100	84,9
2026	85,4	84,4	94,9	99,7	43,7	35,7	100	86,2
Glob.	85,3	83,2	91,1	86,6	71,4	62,1	97,3	65,6

D.3 — Les 6 contrôles qualité (global).

Contrôle	Chambre	Sénat
(a) dates cohérentes (<code>disclosure ≥ transaction</code>)	99,8 %	100,0 %
(b) dépôts ≤ 45 j (délai médian 28 j)	86,9 %	84,9 %
(c) montants — médiane / quartile haut	8 000 \$ / 32 500 \$	
(d) déposants (≥ 10 trades / ≥ 3 ans)	320 (206 / 150)	
(e) achats sans sortie déclarée	22,5 %	39,6 %
(f) validation Quiver — scope « les deux »	86,2 %	99,4 %

D.4 — Validation Quiver par scope, par an (couverture % = trades Quiver retrouvés; `common/quiver_scopes.py`).

An	Chambre			Sénat		
	élec.	OCR	les deux	élec.	OCR	les deux
2020	99,7	61,3	85,1	99,4	—	99,4
2021	99,6	56,0	88,6	98,0	—	98,0
2022	98,2	68,5	80,9	99,4	—	99,4
2023	96,0	67,1	81,0	99,7	0,0	99,7
2024	92,0	78,8	85,8	99,6	0,0	99,6
2025	99,2	61,2	91,3	99,8	—	99,8
2026	98,6	85,3	90,6	100,0	—	100,0
Glob.	98,2	68,3	86,2	99,4	0,0	99,4

Le scope OCR du Sénat est « — » (Quiver ne possède aucun de ces trades : ce sont surtout des munis non cotées). Le scope « les deux » est l'*union dédoublée*, pas la somme.

D.5 — Décomposition Quiver par type d'actif et par cluster (toutes années ; 07g/07h).

Chambre — type d'actif	exact	date ≠	no_match	non_equity	Quiver a %
Stock	47 475	9 301	8 105	2 796	87,5
Mutual Fund	130	92	715	575	23,7
Gov Security	5	0	23	2 259	17,9
Cluster de scan	exact	date ≠	no_match	non_equity	date juste %
A — tapé droit	3 784	630	600	943	85,7
B — tapé couché	18 558	8 653	7 706	7 234	68,2
C — manuscrit	100	169	493	100	37,2

$Quiver\ a\ \% = (exact + date \neq) / actions$; $date\ juste\ \% = exact / (exact + date \neq)$. Sénat : les **Stock** sont couverts à 84,2 %; l'OCR y est surtout du non-coté (Municipal Security, Corporate Bond) → hors périmètre Quiver.

D.6 — Fiabilité des dates OCR (date_confidence, lignes OCR seulement).

An	Chambre (OCR)		Sénat (OCR)	
	plausible %	implausibles	plausible %	implausibles
2020	98,6	120	96,1	9
2021	79,6	1 389	97,5	6
2022	96,4	394	99,5	0
2023	99,2	50	100,0	0
2024	96,5	220	97,6	0
2025	98,1	118	99,0	3
2026	100,0	0	99,7	0
Tot.	95,3	2 291	98,5	18

Le creux 2021 Chambre est entièrement Diana Harshbarger (2 documents manuscrits denses).

D.7 — Clusters OCR & concentration.

Clusters Chambre (census 547 scans)	Concentration des déposants OCR
A — tapé droit : 74 docs	Chambre : Khanna 30 862 (62,9 %), McCaul 10 876 (22,2 %),
B — tapé couché : 322 docs	Harshbarger 3 514 (7,2 %) → top 3 = 92,4 %, top 10 = 99,0 %
C — manuscrit : 151 docs (exclu)	Sénat : Blumenthal 1 233 (73 %), Boozman 379, Burr 52, Feinstein 14, Fetterman 2

D.8 — Statut de validation par déposant (Chambre, crosscheck).

Statut	dépôts	transactions	dont OCR
quiver_validable (Quiver le couvre)	212	78 530	47 004
ocr_unique (Quiver = 0, OCR ≥ 50 %)	13	1 957	1 949
digital (peu / pas d'OCR)	31	1 155	13

Identité : Chambre 256 bioguides / 275 noms (99,99 %), Sénat 64 / 67 (100 %). Golden (zéro écart) : **125** fichiers Chambre, **76** Sénat. **Total : 90 487 transactions**, 2020–2026.

11.5 Annexe E — reproduire / lancer

```
# Installation
python3.12 -m venv .venv && ./venv/bin/pip install -e .

# Pipeline complet (reseau + API Vision requis) ; --dry-run = voir la sequence
./venv/bin/python -m common.pipeline --years 2020-2026
./venv/bin/python -m common.pipeline --years 2024 --dry-run

# Rapport de qualite (offline, lecture seule des FINAL)
#   -> docs/RAPPORT_QUALITE.md + docs/quality/*.png
./venv/bin/python -m common.quality

# Filet "zero changement" (doit afficher ZERO ECART)
./venv/bin/python tests/regression/check_golden.py           # Chambre, 125
./venv/bin/python tests/regression/senate_check_golden.py    # Senat,    76
./venv/bin/python tests/regression/test_senate_repro.py      # reproductions
```

Document dérivé et vérifié sur le code source réel et les tables FINAL du dépôt. Toute divergence avec une autre documentation doit être tranchée en faveur du code et des données. Quiver = vérification externe, jamais réinjecté.